Lanhai-driver

Standard SDK User Guide

Jinhua Lanhai Photoelectric Technology Co., Ltd.

https://www.pacecat.com/

Contents

1. Introduction 1
2. SDK File Organization 1
3. SDK Development Guide 1
3.1 SDK Composition 2
3.2 Header File Introduction 2
3.3 Main SDK Function Interface Descriptions 3
3.4 Notes 6
4. Configuration File Description 7
5. SDK Compilation 8
6. Example Program 8

1. Introduction

This document applies to the standard open-source version of the LIDAR SDK. The SDK currently works in Windows and Linux environments. It supports compilation with VC++6.0 or later (g++ 98/gcc99).

2. SDK File Organization

The SDK file structure is shown below (cmake project):

```
lanhai-driver-branch
  .git
  build
  config
  docs
  example
  sdk
  web
  .gitignore
  cmake_install.cmake
  CMakeLists.txt
  Makefile
  readme.md
```

Config: Configuration file parameters for commonly used lidar models

example: Test examples, shared by Windows/Linux here

Docs: Document directory

Sdk: Core code (integrated demo)

Web: Web resources

CMakeLists.txt

Readme.md

3. SDK Development Guide

3.1 SDK Integration Instructions

The SDK is provided as source code. To integrate it, add the sdk folder and include the single header file standard_interface.h. For details, see main.cpp in the demo folder.

For point cloud data reading, synchronized timestamp printing, runtime information printing, etc., refer to:

```
void CallBackMsg(int msgtype, void *param,int length)
```

Parameter 1: Custom message type
Parameter 2: Callback data
Parameter 3: Length of callback data

Main steps:

1. The main thread adds the configuration file data through addLidarByPath, which returns a unique ID.

2. The main thread sets the callback function of each lidar worker thread through setCallBackPtr. Refer to CallBackMsg.

3. The main thread opens the lidar with the specified ID through openDev and starts the worker thread (required), web visualization thread (optional), and heartbeat thread (optional).

4. The worker thread returns the specific point cloud information through the callback function.

5. The main thread calls the interfaces in standard_interface.h to read/set other functions. See the interfaces below for details.

3.2 Header File Introduction

Data structures: data.h

Error definition header file: error.h

Common interfaces: Global.h

Data processing: LidarDataProcess.h

Separate serial-port functions (linux): Uart.c

Service classes:

LidarCheckService.h: Detects available serial/network/protection-zone lidar devices on the current subnet.

LidarWebService.h: Web service for browser-side control of the lidar program, data acquisition, and other functions.

Third-party libraries:

cJSON

mongoose

Layui

echarts

jquery

Provided externally:

standard_interface.h: SDK header provided for users

3.3 Main SDK Function Interface Descriptions

This section mainly explains standard_interface.h/cpp, which is provided directly for customer integration.

addLidarByPath

```
int addLidarByPath(const char *cfg_file_name)
```

Function parameters: 1. Absolute path + name of configuration file [IN]

Function purpose: Add a lidar through a configuration file

Return value: Lidar ID

Other notes:

delLidarByID

```
bool delLidarByID(int ID);
```

Function parameters: 1. Lidar ID [IN]

Function purpose: Delete the specified lidar by ID

Return value: True/false

Other notes:

setCallBackPtr

```
void setCallBackPtr(int ID,printfMsg ptr);
```

Function parameters: 1. Lidar ID [IN] 2. Callback pointer [IN]
typedef void (*printfMsg)(int, void*,int);

Function purpose: Set the corresponding callback pointer according to the lidar ID

Return value:

Other notes:

openDev

```
bool openDev(int ID);
```

Function parameters: 1. Lidar ID [IN]

Function purpose: Open the lidar device and run a subprocess to process data separately

Return value: True/false

Other notes: A success message is printed if it runs successfully

StopDev

```
void StopDev(int ID);
```

Function parameters: 1. Lidar ID [IN]

Function purpose: Close the lidar device and its child threads

Return value:

Other notes:

GetDevInfo

```
bool GetDevInfo(int ID,EEpromV101 *eepromv101);
```

Function parameters: 1. Lidar ID [IN] 2. Structure receiving device parameters [OUT]

Function purpose: Get lidar parameters

Return value: Ture/false

Other notes: All data can be obtained for network and protection-zone models.
Only the serial number is obtained for serial models.

ControlDrv

```
bool ControlDrv(int ID, int num, char *cmd);
```

Function parameters: 1. Lidar ID [IN] 2. Command length [IN] 3. Command [IN]

Function purpose: Control lidar operation

Return value: True/false

Other notes: Commands include the following 4 types:
LSTARH: Start running
LSTOPH: Stop running
LRESTH: Run again

getVersion

```
const char* getVersion();
```

Function parameters: None

Function purpose: Get the SDK version number

Return value: Version number

Other notes: None

ZoneSection

```
bool ZoneSection(int ID, char section);
```

Function parameters: 1. Lidar ID 2. Selected protection zone

Function purpose: Switch protection zone

Return value: True/false

Other notes: Protection zone range: 0-9,A-F

SetUDP

```
bool SetUDP(int ID, char* ip, char* mask, char* gateway, int port)
```

Function parameters: 1. Lidar ID 2. Lidar IP 3. Mask 4. Gateway 5. Port

Function purpose: Set the network of the target lidar

Return value: True/false

Other notes: After changing the lidar IP, the socket must be set again to receive subsequent data.

SetDST

```
bool SetDST(int ID, char* ip, int port);
```

Function parameters: 1. Lidar ID 2. IP for lidar data uploads 3. Port

Function purpose: Set the data upload address of the target lidar

Return value: True/false

Other notes:

SetRPM

```
bool SetRPM(int ID,int RPM);
```

Function parameters: 1. Lidar ID 2. Rotation speed

Function purpose: Set the rotation speed of the lidar

Return value: True/false

Other notes:

SetTFX

```
bool SetTFX(int ID,bool tfx);
```

Function parameters: 1. Lidar ID 2. Whether upload is fixed

Function purpose: Set fixed upload

Return value: True/false

Other notes: Disabled by default

SetDSW

```
bool SetDSW(int ID,bool dsw);
```

Function parameters: 1. Lidar ID 2. Whether to remove trailing points

Function purpose: Set trailing-point removal

Return value: True/false

Other notes: Enabled by default

SetSMT

```
bool SetSMT(int ID,bool smt);
```

Function parameters: 1. Lidar ID 2. Whether data smoothing is enabled

Function purpose: Set data smoothing

Return value: True/false

Other notes: Enabled by default

SetPST

```
bool SetPST(int ID,int mode);
```

Function parameters: 1. Lidar ID 2. Data upload type

Function purpose: Set SetPST

Return value: True/false

Other notes: Data upload type: 0 No data 1 Data 2 Alarm 3 Data + alarm

SetDID

```
bool SetDID(int ID,unsigned int number);
```

Function parameters: 1. Lidar ID 2. Lidar number

Function purpose: Set the lidar number

Return value: True/false

Other notes: Different from the lidar ID

getLidarsList

```
std::vector<DevConnInfo> getLidarsList()
```

Function parameters:

Function purpose: Get heartbeat packet data from network lidar devices and currently available serial lidar devices

Return value: Array of lidar information structures

Other notes:

3.4 Notes

The SDK is developed in C/C++. Before using the lanhai-driver SDK, developers are advised to understand the lidar communication protocol (uart/udp) and operating modes (conditions of normal hardware operation, including indicator lights, etc.). Refer to the user manual for the specific model for details, or consult the after-sales instructions for frequently asked questions.

4. Configuration File Description

connect

type - Lidar type. uart: Serial port; udp: Network model; udp_uart: Virtual serial port

connectArg - USB port number of serial lidar: Win:com1; Linux:/dev/ttyUSB0; Linux VPC:/dev/ttyACM0. Target lidar IP, e.g. 192.168.158.98

connectArg2 - Serial lidar baud rate, e.g. 768000. Access port of target lidar, e.g. 6543

local_port - Host receive port. Example: 6888

is_group_listener - Whether to enable broadcast. 0/1

group_ip - Broadcast IP. 224.0.0.99

data

output_360 - Whether to output once per 360 degrees. 0: Partial sector; 1: Full sector

from_zero - Whether to output from zero degrees. 0/1: 0 -180 degrees to 180 degrees; 1 0 degrees to 360

service_port - Open port of the local Web service. Example: 8888

is_open_service - Whether to start the web service. 0/1

error_circle - Count erroneous revolutions that meet the requirement (number of points with distance 0 / total number exceeds the specified coefficient). Default: 3

error_scale - Proportional coefficient: points with distance 0 : total points. Default: 0.9

filter

filter_open - Filter activation switch. 1

max_range - Maximum distance at which filtering takes effect. 20

min_range - Minimum distance at which filtering takes effect. 0.5

max_range_difference - Physical range for determining outliers. 0.1

filter_window - Index range for determining outliers. 1

get

model - Query model. -1 Do not execute; >=0 Execute

version - Query hardware version number. -1 Do not execute; >=0 Execute

set

unit_is_mm - Whether millimeters are the data unit. -1/0/1 Do not execute/No/Yes

with_confidence - Whether data includes intensity. -1/0/1 Do not execute/No/Yes

with_smooth - Whether data is smoothed. -1/0/1 Do not execute/No/Yes

with_deshadow - Whether trailing points are removed. -1/0/1 Do not execute/No/Yes

resample - Angular resolution. -1/0/1/xxx Do not execute/Non-fixed/Fixed angular resolution/Specific fixed value

rpm - Rotation speed. -1/xx Do not execute/Specific rotation speed

direction - Rotation direction. -1/0/1 Do not execute/Clockwise/Counterclockwise. Effective only for some lidar devices

alarm_msg - Alarm information upload. -1/0/1 Do not execute/Data only/Data plus alarm

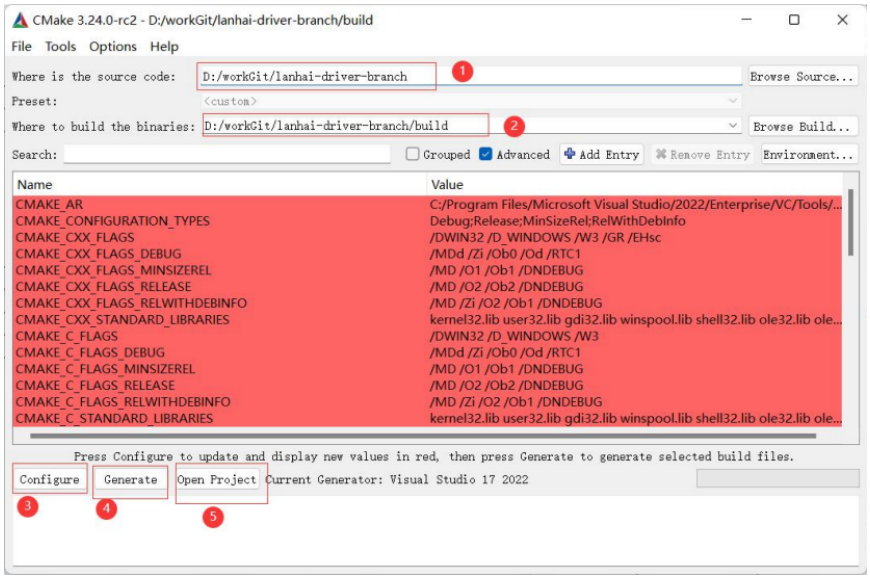
ats - Connection method. -1/0/1 Do not execute/Network/USB

with_start - Send the start-rotation command by default at startup. -1/0/1 Do not execute/No/Yes

5. SDK Compilation

Windows:

5.1 Widows: Download the cmake tool, then use cmake-gui to generate the specified VS project. VS2022 is used here as an example (a relatively recent cmake version is required).



5.2 VS project: Solution -> Set demo as Startup Project

5.3 Project -> Properties -> Debugging: Set the command arguments (the lidar configuration file to use) and working directory (fixed).

,

demo Properties

Configuration: Debug Platform: Active(Win32) Configuration Manager...

Configuration Properties: General; Debugging; VC++ Directories; C/C++; Linker; Manifest Tool; XML Document Generator; Browse Information; Build Events; Custom Build Step; Custom Build Tool; Code Analysis.

Debugger to launch: Local Windows Debugger

Command: $(TargetPath)

Command Arguments: ./config/LSS-40D-C30E.txt

Working Directory: $(ProjectDir)./

Attach: No

Debugger Type: Auto

Environment:

Merge Environment: Yes

SQL Debugging: No

Amp Default Accelerator: WARP Software Accelerator

Command: The command to execute when debugging.

OK Cancel Apply(A)

5.4 Build -> Build Solution

5.5 Debug -> Start Running

Linux

```
5.6 cmake CMakeList.txt
5.7 make
5.8 ./demo config/xxxx.txt
```

Note: For a serial device in a virtual machine, also run sudo chmod 777 /dev/ttyUSB* to grant permissions to the device.

6. Example Program

For source code details, see main.cpp in the example directory.

For executable file details, see the tools directory.

6.1 Multiple Lidar Configuration

Pass the relative path (absolute path) of each configuration file here.

For example: ./demo config/LDS-E310-E.txt config/LDS-50C-C30E.txt. The specific configuration information follows.

```
type:udp
port:com7
baud_rate:768000
lidar_ip:192.168.0.75
lidar_port:6543
lidar_ip1:192.168.0.98
lidar_port2:6543
local_port:6669
raw_bytes:3
unit_is_mm:1
with_confidence:1
with_checksum:1
with_smooth:1
with_deshadow:1
resample:1
rpm:600
output_scan:1
output_360:1
from_zero:1
output_file:C:\qwer.txt
is_group_listener:0
group_ip:224.0.0.99
service_port:8889
```

```
type:udp
port:com7
baud_rate:768000
lidar_ip:192.168.0.98
lidar_port:6543
local_port:6669
raw_bytes:3
unit_is_mm:1
with_confidence:1
with_checksum:1
with_smooth:1
with_deshadow:1
resample:200
rpm:600
output_scan:1
output_360:1
from_zero:0
output_file:C:\123.t
is_group_listener:0
group_ip:224.0.0.99
service_port:8889
is_open_service:1
```

Note: local_port and service_port in the configuration files must be different.

After successful startup, two success messages are printed. If either one fails, the process exits immediately.

```
----> start udp 192.168.0.75:6543 udp 576
send command : 'LSTARH'
set LiDAR LSTARH OK
send command : 'LXVERH'
set LiDAR LXVERH OK V301 221101
Lidar is open success
Please control it through a browser and enter the default address: http://localhost: 8889
angle sum 3510, drop 38 fans 2925 points
getLidarData recv MSG
GetDevInfo recv MSG
Device: number 1; serial LH6401210400011; model LDS-E310-E
IP 192.168.0.75; mask 255.255.255.0; gateway 192.168.0.1;
default target IP 192.168.0.49; default target UDP port 6669; external UDP port 6543
Speed 600; motor startup 1000; FIR filter 1; turns 1; resolution 1; auto-upload 1;
fixed upload 1; smoothing 1; trailing-point removal 1; network heartbeat 0; IO polarity 0
Smoothing coefficient 20; activation zone 0; upload data type 1
----> start udp 192.168.0.98:6543 udp 620
send command : 'LSTARH'
set LiDAR LSTARH OK
send command : 'LXVERH'
set LiDAR LXVERH OK V301 221104
send command : 'LSRES:0200H'
set LiDAR resample 200 OK
Lidar is open success
Please control it through a browser and enter the default address: http://localhost: 8888
angle sum 1800, drop 10 fans 900 points
getLidarData recv MSG
angle sum 3420, drop 19 fans 1710 points
GetDevInfo recv MSG
Device: number 1; serial GS230103012; model LDS-50C-C30E
IP 192.168.0.98; mask 255.255.255.0; gateway 192.168.0.1;
default target IP 192.168.0.48; default target UDP port 6668; external UDP port 6543
Speed 600; motor startup 1000; FIR filter 3; turns 2; resolution 0; auto-upload 1;
fixed upload 0; smoothing 1; trailing-point removal 1; network heartbeat 0; IO polarity 0
Smoothing coefficient 14; activation zone 15; upload data type 1
```

To view the web display, open two tabs. Their ports correspond to service_port in the configuration files.

http:://localhost:8888
http:://localhost:8889

6.2 Warning When a Full Revolution of Lidar Data Is 0

```
Notepad.txt | LDS-50C-C30E.txt | LDS-50C-2.txt
type:uart
port:com10
baud_rate:500000
lidar_ip:192.168.0.208
lidar_port:6543
local_port:6668
raw_bytes:3
unit_is_mm:1
with_confidence:1
with_checksum:1
with_smooth:1
with_deshadow:1
resample:400
rpm:600
output_scan:1
output_360:0
from_zero:0
output_file:/tmp/udp_data2.txt
is_group_listener:0
group_ip:224.0.0.99
service_port:8888
is_open_service:1
error_circle:3
error_scale:0.9
```

Add to the configuration file:

error_circle: Here this means that points with distance 0 occur in 3 consecutive revolutions and exceed the specified proportion of the number of points.

error_scale: This is the custom coefficient for zero distance, namely total number of points * specified coefficient = number of points used for the judgment.

```
// Print by passing a callback pointer
void CallBackMsg(int msgtype, void* param)
{
    // Real-time radar data return
    if (msgtype == 1) {...}
    // Real-time protection-zone data return
    else if (msgtype == 2) {...}
    // Radar error information
    else if (msgtype == 3)
    {
        char* result = (char*)param;
        INFO_PR("Error Info : %s\n", result);
    }
    // Get lidar timestamp printing information
    else if (msgtype == 4) {...}
}
```

At the outermost level, demo.cpp prints errors returned by the callback function. An error is returned only when the distances of all received points in one sector/one revolution are 0 (sector/revolution depends on output_360).

```
LidarDataProcess.cpp | standard_interface.h | LidarDataProcess.h | main.cpp
lidar_thread_proc_udp(void *param)
    // Process the sector
    UserAPI::fan_data_process(dat, cfg->output_file, tmp);
}
else
{
    // Full data, full sector
    UserAPI::whole_data_process(dat, cfg->from_zero,
        cfg->collect_angle, cfg->output_file, tmp, whole_datas);
}
if (tmp.N > 0)
{
    if (checkPointsLengthZero(tmp, cfg->error_scale))
        error_num++;
    else
        error_num = 0;
    if (cfg->error_circle <= error_num)
    {
        char tmp_str[128] = {};
        sprintf(tmp_str, "%s %d There are many points with a distance of 0 in the current radar operation",
            cfg->lidar_ip, cfg->lidar_port);
        ((void(*)(int, void*))cfg->callback)(3, tmp_str);
        error_num = 0;
    }
    strcpy(tmp.ip, cfg->lidar_ip);
    tmp.port = cfg->lidar_port;
    ((void(*)(int, void*))cfg->callback)(1, &tmp);
    memcpy(&cfg->pointdata, &tmp, sizeof(PointData));
}
```

Internal adjustment: When information for one revolution/one sector is obtained, all point distances are checked. If the number of points meeting the requirement reaches the specified coefficient and continues for the specified number of revolutions, an error is returned through the callback function. If points meeting the requirement occur in between, the accumulated error revolution count is reset.

6.3 Obtaining Various Lidar Data (Integration Use)

For details, refer to the CallBackMsg callback function in main.cpp. The specific returned data includes:

- Point cloud data
- Alarm data: Add alarm_msg:1 to the configuration file (enable alarm data upload)
- Error messages
- Timestamp printing

For lidar global parameters, examine the PrintMsg function.

7. Control Command Table

Description: These are the meanings of hardware commands involved in the SDK. Not all models support all the following commands. Consult the user manual for the particular lidar model for supported commands.

Command - Description

LSTARH - Start running

LSTOPH - Stop running

LRESTH - Run again

LXVERH - Get hardware information

LMDCMH - Set lidar point cloud data unit to CM

LMDMMH - Set lidar point cloud data unit to MM

LNCONH - Enable lidar intensity data

LOCONH - Disable lidar intensity data

LFFF0H

Enable trailing-point removal

LFFF1H

Disable trailing-point removal

LSSS0H

Enable data smoothing

LSSS1H

Disable data smoothing

LSRES:000H

Set default angular resolution

LSRES:001H

Set corrected angular resolution

LSRPM:%04dH

Set rotation speed. Range 0450-1200. Example: LSRPM:0450H

LSERR:+%dH

Set angular deviation. Range -99-+99. Examples: LSERR:-23H; LSERR:+23H

LSUDP:%sH

Set UDP combined information: lidar IP address, subnet mask, gateway, service port number.
Example: LSUDP:192.168.158.091 255.255.255.000 192.168.158.001 05000H

LSDST:%sH

Set the IP address and port number for receiving lidar information. LSDST:192.168.158.043 12300H

LSUID:%sH

Set machine serial number. Example: LSUID:201812030001H

LSSMT:%dH

Set data smoothing. LSSMT:1H Enable; LSSMT:0H Disable

LSDSW:%dH

Set trailing-point removal. LSDSW:1H Enable; LSDSW:0H Disable

LSDID:%dH

Set device ID. LSDID:xxxH

LSATS:%dH

Automatic upload at power-on. LSATS:xH 1/0 Enable/Disable

LSTFX:%dH

Fixed upload address. LSTFX:xH 1/0 Enable/Disable

LSPST:%dH

Data/alarm upload type. LSPST:xH 0: None; 1: Data; 2: Alarm; 3: Data + alarm